

Cursor IDE 完全操作手冊

初級使用者指南 | AI程式碼協助

目錄

1. 認識 Cursor

2. 介面介紹

3. 基礎操作

4. AI 程式碼協助功能

5. 快捷鍵速查表

6. 常見問題

認識 Cursor

什麼是 Cursor？

Cursor 是一款**現代化的 AI 驅動程式碼編輯器**，結合了強大的 AI 助手（如 GPT-4、Claude），幫助開發者更高效地：

- 📝 自動補全程式碼
  - 🤖 利用 AI 重構與優化程式碼
  - 🔍 快速定位和修復錯誤
  - 📄 生成測試和文檔
  - 💡 獲得智慧提示和建議

為什麼選擇 Cursor？

- ✅ 直觀的使用者介面
  - ✅ 強大的 AI 整合
  - ✅ 支援所有主流程式語言
  - ✅ 完整的版本控制（Git）整合
  - ✅ 輕量且高效

介面介紹

🧩 整體布局





## 1 頂部菜單欄

| 菜單       | 主要功能            |
|----------|-----------------|
| File     | 新建檔案、開啟專案、保存、關閉 |
| Edit     | 複製、貼上、查找、替換     |
| View     | 切換版面、調整字型大小     |
| Terminal | 開啟終端機執行命令       |
| Help     | 查詢幫助文檔、快捷鍵      |

## 2 左側邊欄（重要！）

邊欄上方有 5 個主要圖示：

### Explorer（檔案總管）

- 瀏覽你的專案檔案和資料夾
- 右鍵點擊可建立新檔案、新資料夾
- 點擊開啟、編輯、刪除檔案

### Search（搜尋）

- 在全專案中搜尋字串或正則表達式
- 支援大小寫敏感、全字匹配、正則模式

### Source Control（版本控制）

- 檢視 Git 變更
- 提交（commit）程式碼
- 推送（push）、拉取（pull）遠端倉庫
- 檢視程式碼差異

### Run & Debug（執行與除錯）

- 執行你的程式
- 設置斷點進行除錯
- 檢視變數、堆疊跟蹤

### Extensions（擴展）

- 安裝程式語言支援

- 安裝主題和插件
- 增強編輯器功能

### 3 編輯區（中心）

- **多標籤編輯**：同時開啟多個檔案，點擊標籤切換
- **行號**：左側顯示行號，方便定位
- **代碼高亮**：自動根據語言提供色彩高亮
- **迷你地圖**：右側顯示檔案預覽

### 4 底部狀態列

| 項目    | 說明         |
|-------|------------|
| 行:列   | 游標位置       |
| 語言    | 檔案的程式語言    |
| Git分支 | 目前的Git分支名稱 |
| 錯誤/警告 | 代碼問題數量     |

## 基礎操作

### ✓ 第1步：開啟或建立專案

#### 方法 A：開啟現有專案

1. 點擊 **File** → **Open Folder**
2. 選擇你的專案資料夾
3. 點擊「打開」

#### 方法 B：新建專案

1. 點擊 **File** → **New Window**
2. 或直接拖曳資料夾到 Cursor 視窗

### ✓ 第2步：建立新檔案

#### 方法1：使用 Explorer

1. 在左側 Explorer 中右鍵點擊資料夾
2. 選擇「New File」
3. 輸入檔名（含副檔名，如 `app.js`、`index.html`）

#### 方法2：使用快捷鍵

```
Ctrl + N   (Windows/Linux)
Cmd + N    (Mac)
```

### ✅ 第3步：編寫程式碼

1. 在編輯區輸入程式碼
2. Cursor 會自動高亮語法
3. 保存檔案：**Ctrl + S**

### ✅ 第4步：執行程式

1. 點擊左側「Run & Debug」圖示
2. 選擇執行環境（Node.js、Python等）
3. 按「執行」按鈕（綠色三角形）
4. 輸出結果顯示在下方終端

### ✅ 第5步：提交到 Git

1. 點擊左側「Source Control」圖示
2. 檢視你的變更
3. 點擊「+」號暫存檔案
4. 在提交訊息框輸入說明
5. 按 **Ctrl + Enter** 提交

---

## AI 程式碼協助功能

### 功能1：智慧補全（Auto-completion）

#### 自動觸發

- 當你輸入時，Cursor AI 會自動建議程式碼片段
- 建議會以灰色文字浮現在編輯器中

#### 接受建議

按 Tab → 接受整個建議  
按 Escape → 拒絕建議  
按 Ctrl+Space → 手動觸發補全

#### 範例

```
// 你輸入：  
function greet(  
  
// Cursor 建議：  
function greet(name) {  
  console.log("Hello, " + name);  
}
```

## 功能2：聊天協助（Inline Chat）

### 開啟聊天

按 Ctrl + K（Windows/Linux）

按 Cmd + K（Mac）

### 使用方式

1. 游標停在程式碼位置
2. 按快捷鍵開啟聊天框
3. 輸入你的問題或指令
4. 按 Enter 發送

### 常見提示詞（Prompts）

| 指令 | 說明        | 範例                      |
|----|-----------|-------------------------|
| 解釋 | 解釋程式碼的含義  | "Explain this function" |
| 修復 | 修復程式碼中的錯誤 | "Fix this bug"          |
| 優化 | 優化程式碼效能   | "Optimize for speed"    |
| 測試 | 生成測試代碼    | "Write Jest tests"      |
| 重構 | 改進程式碼結構   | "Refactor this code"    |
| 轉換 | 轉換程式語言    | "Convert to TypeScript" |

### 範例對話

【你】Explain what this function does

function factorial(n) {  
 return n <= 1 ? 1 : n \* factorial(n - 1);  
}

【Cursor】This function calculates the factorial of a number using recursion. If n is 1 or less, it returns 1 (base case). Otherwise, it multiplies n by the factorial of n-1.

## 功能3：代碼生成（Code Generation）

### 建立新程式碼片段

1. 按 **Ctrl + K** 開啟聊天
2. 輸入你要建立的內容
3. Cursor 會直接在編輯器中生成程式碼

## 範例

【你】 Create a React component for a login form

【Cursor 生成】

```
import React, { useState } from 'react';

export default function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    // 處理登入邏輯
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

## 功能4：代碼編輯（Code Edit）

### 選擇程式碼後編輯

1. 高亮選中你要修改的程式碼
2. 按 **Ctrl + K** 開啟聊天
3. 輸入修改指令
4. Cursor 會直接修改選中的程式碼

## 範例

```
// 原始程式碼
function add(a, b) {
  return a + b;
}
```

```
}

// 【你】 Add error handling and type checking

// 【Cursor 修改後】
function add(a: number, b: number): number {
  if (typeof a !== 'number' || typeof b !== 'number') {
    throw new Error('Both parameters must be numbers');
  }
  return a + b;
}
```

## 功能5：生成測試（Test Generation）

### 為你的函數生成測試

1. 高亮函數代碼
2. 按 **Ctrl + K**
3. 輸入：**Write Jest tests for this function**
4. Cursor 自動生成測試程式碼

### 範例

```
// 原始函數
function isEven(num) {
  return num % 2 === 0;
}

// 【生成的測試】
describe('isEven', () => {
  test('should return true for even numbers', () => {
    expect(isEven(2)).toBe(true);
    expect(isEven(4)).toBe(true);
  });

  test('should return false for odd numbers', () => {
    expect(isEven(1)).toBe(false);
    expect(isEven(3)).toBe(false);
  });
});
```

## 功能6：錯誤修復（Error Fixing）

### 修復錯誤訊息

1. 複製你遇到的錯誤訊息
2. 按 **Ctrl + K** 開啟聊天
3. 貼上錯誤訊息，問「How do I fix this?」

## 4. Cursor 會給出解決方案

### 範例

【錯誤】 `TypeError: Cannot read property 'name' of undefined`

【你】 Why is this error happening and how do I fix it?

【Cursor】 This error means you're trying to access the 'name' property of a variable that is undefined. This usually happens when:

1. An object isn't initialized before use
2. A function doesn't return expected data
3. An API call failed

To fix it:

- Check if the object exists before accessing its properties
- Use optional chaining: `object?.name`
- Add null checks

---

## 功能7：代碼文檔（Documentation）

### 生成註解和文檔

1. 高亮函數或類別
2. 按 `Ctrl + K`
3. 輸入：`Add JSDoc comments` 或 `Write documentation`
4. Cursor 自動補充詳細註解

### 範例

```
// 原始函數
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}

// 【添加文檔後】
/**
 * 計算項目列表的總價格
 * @param {Array<{price: number}>} items - 包含價格的項目陣列
 * @returns {number} 所有項目的總價格
 * @example
 * const items = [{price: 10}, {price: 20}];
 * calculateTotal(items); // 返回 30
 */
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}
```

# 快捷鍵速查表

## 編輯快捷鍵

| 快捷鍵  | 功能     | Windows/Linux | Mac     |
|------|--------|---------------|---------|
| 保存   | 保存當前檔案 | Ctrl + S      | Cmd + S |
| 全選   | 選中全部文字 | Ctrl + A      | Cmd + A |
| 複製   | 複製選中文字 | Ctrl + C      | Cmd + C |
| 貼上   | 貼上文字   | Ctrl + V      | Cmd + V |
| 撤銷   | 撤銷上一步  | Ctrl + Z      | Cmd + Z |
| 重做   | 重做撤銷操作 | Ctrl + Y      | Cmd + Y |
| 新檔案  | 建立新檔案  | Ctrl + N      | Cmd + N |
| 開啟檔案 | 開啟檔案   | Ctrl + O      | Cmd + O |

## AI 快捷鍵

| 快捷鍵  | 功能       | Windows/Linux | Mac         |
|------|----------|---------------|-------------|
| 聊天   | 開啟 AI 聊天 | Ctrl + K      | Cmd + K     |
| 補全   | 手動觸發補全   | Ctrl + Space  | Cmd + Space |
| 接受建議 | 接受補全建議   | Tab           | Tab         |
| 拒絕建議 | 拒絕補全建議   | Escape        | Escape      |

## 導航快捷鍵

| 快捷鍵  | 功能      | Windows/Linux    | Mac             |
|------|---------|------------------|-----------------|
| 查找   | 在檔案中查找  | Ctrl + F         | Cmd + F         |
| 替換   | 查找並替換   | Ctrl + H         | Cmd + H         |
| 轉到行  | 跳至指定行   | Ctrl + G         | Cmd + G         |
| 打開命令 | 打開命令面板  | Ctrl + Shift + P | Cmd + Shift + P |
| 側邊欄  | 切換側邊欄   | Ctrl + B         | Cmd + B         |
| 終端   | 開啟/關閉終端 | Ctrl + `         | Cmd + `         |

## 執行和除錯

| 快捷鍵 | 功能   | Windows/Linux  | Mac           |
|-----|------|----------------|---------------|
| 執行  | 執行程式 | F5 或 Ctrl + F5 | F5 或 Cmd + F5 |

| 快捷鍵 | 功能       | Windows/Linux | Mac      |
|-----|----------|---------------|----------|
| 除錯  | 開始除錯     | F5            | F5       |
| 繼續  | 繼續執行     | F5            | F5       |
| 步進  | 單步執行     | F10           | F10      |
| 步入  | 步入函數     | F11           | F11      |
| 斷點  | 在當前行設置斷點 | Ctrl + F9     | Cmd + F9 |

## 常見問題

**? Q1：**Cursor 支援哪些程式語言？

**A：**Cursor 支援幾乎所有主流程式語言，包括：

- JavaScript / TypeScript
- Python
- Java
- C / C++
- Go
- Rust
- PHP
- Ruby
- SQL
- HTML / CSS
- 以及更多...

透過安裝擴展可以添加更多語言支援。

**? Q2：**AI 補全不出現怎麼辦？

**A：**嘗試以下方法：

1. 檢查 AI 服務是否已啟用（底部狀態列）
2. 按 **Ctrl + Space** 手動觸發補全
3. 檢查你的網路連接
4. 重啟 Cursor
5. 在設定中檢查補全是否被禁用

**? Q3：**如何更改主題？

**A：**

1. 按 **Ctrl + ,**（或 **Cmd + ,**）打開設定
2. 搜尋「Color Theme」
3. 點擊下拉選單選擇主題

4. 常見主題：Dark+ 、Light+ 、Dracula 等

---

### ? Q4：如何建立 Git 倉庫？

A：

1. 點擊左側「Source Control」圖示
2. 點擊「Initialize Repository」
3. 選擇資料夾
4. 完成初始化

或在終端輸入：

```
git init
```

---

### ? Q5：如何更改字型大小？

A：

1. 按 **Ctrl + ,** 打開設定
  2. 搜尋「Font Size」
  3. 輸入數字（預設 14）
  4. 立即生效
- 

### ? Q6：Cursor 會記錄我的程式碼嗎？

A：

- 取決於你選擇的 AI 模型和隱私設定
  - 強烈建議檢查設定中的隱私政策
  - 對於敏感或機密代碼，考慮使用本地部署模型
  - 查看官方隱私文檔：[cursor.so/privacy](https://cursor.so/privacy)
- 

### ? Q7：如何安裝擴展？

A：

1. 點擊左側「Extensions」圖示（拼圖圖標）
2. 在搜尋框中輸入擴展名
3. 點擊搜尋結果
4. 點擊「Install」安裝
5. 安裝完成後會提示重載窗口

推薦擴展：

- Prettier（代碼格式化）

- ESLint (JavaScript 檢查)
- Thunder Client (API 測試)
- GitLens (Git 增強)

 Q8：AI 聊天時如何提供更多上下文？

A：

1. 在開啟聊天前選擇相關程式碼
2. 你也可以在聊天中引用整個檔案
3. 提供更多上下文 → AI 給出更準確的建議
4. 可以在一個對話中提出多個相關問題

 Q9：如何禁用某些功能？

A：

1. 按 `Ctrl + ,` 打開設定
2. 搜尋相應功能名稱
3. 取消勾選或禁用
4. 常見選項：
  - Enable AutoSave (自動保存)
  - Enable IntelliSense (智慧感知)
  - Format on Save (保存時格式化)

 Q10：Cursor 和 VS Code 有什麼區別？

A：

| 特性    | Cursor    | VS Code |
|-------|-----------|---------|
| AI 整合 | ✅ 內建強大 AI | ❌ 需要擴展  |
| 補全建議  | ✅ AI 驅動   | ⚠️ 基礎補全 |
| 代碼生成  | ✅ 完整支援    | ❌ 不支援   |
| 介面    | 清晰直觀      | 功能豐富    |
| 擴展生態  | 較新        | 極其豐富    |
| 學習曲線  | 較低        | 中等      |

 實用資源

官方資源

- 官方網站：[cursor.so](https://cursor.so)
- 文檔：[cursor.so/docs](https://cursor.so/docs)

- **FAQ** : [cursor.so/faq](https://cursor.so/faq)

## 社群資源

- **Discord 社群**：獲得幫助、分享經驗
- **GitHub Issues**：報告問題、建議功能
- **YouTube 教學**：視覺化學習指南

## 學習建議

1.  從基礎操作開始掌握
2.  逐漸學習 AI 功能的各種用途
3.  定期查看快捷鍵，提升工作效率
4.  實踐才是最好的學習方式
5.  根據需要自訂快捷鍵和設定



## 進階建議

### 為初學者的 10 個 Tips

1. **設定自動保存**  
避免丟失程式碼，在設定中啟用 AutoSave
2. **學習常用快捷鍵**  
掌握 5 個最常用的快捷鍵能大幅提升效率
3. **充分利用 AI 聊天**  
AI 聊天不只是補全，還能幫助理解、除錯、優化
4. **定期提交到 Git**  
養成頻繁提交的習慣，方便追蹤變更
5. **使用擴展增強功能**  
安裝語言支援、格式化工具等擴展
6. **調整編輯器設定**  
根據習慣調整字型、主題、縮進等
7. **探索代碼生成**  
用 AI 快速生成樣板代碼，節省時間
8. **為複雜代碼添加註解**  
利用 AI 自動生成文檔和註解
9. **測試驅動開發**  
用 AI 幫助生成單元測試
10. **保持軟體更新**  
定期更新 Cursor 以獲得最新功能和安全補丁

## 使用示例

### 範例 1：建立一個 Todo 應用

#### 第1步：建立專案

```
mkdir todo-app
cd todo-app
```

#### 第2步：建立檔案

1. 在 Cursor 中開啟 todo-app 資料夾
2. 建立 `index.html`、`style.css`、`script.js`

#### 第3步：AI 輔助開發

【你】 Create a todo list application with add and delete functionality

【Cursor 生成完整代碼】

#### 第4步：測試和提交

```
git init
git add .
git commit -m "Initial todo app"
```

---

### 範例 2：優化現有函數

#### 原始函數：

```
function search(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;
  }
  return -1;
}
```

#### 步驟：

1. 高亮程式碼
2. 按 `Ctrl + K`
3. 輸入：「Optimize this search function」
4. Cursor 建議使用二分查找或其他優化方案

---

## 總結

Cursor 是一款功能強大且易於上手的現代化編輯器。通過：

-  理解介面布局
-  掌握基本操作
-  充分利用 AI 功能
-  記住常用快捷鍵

你可以大幅提升編程效率和代碼品質。祝你使用愉快！

---

**最後更新：**2024年7月

**手冊版本：**1.0

**適用版本：**Cursor 最新版本